SVM Classifier

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# Load the dataset
data = pd.read_csv('svm.csv')

data
```

	Length	Weight	Snout Width	Habitat	Species
0	3.4	400	20	Saltwater	Crocodile
1	4.2	500	22	Freshwater	Alligator
2	3.9	450	19	Saltwater	Crocodile
3	4.5	600	24	Freshwater	Alligator
4	2.8	300	21	Saltwater	Crocodile
5	3.6	480	23	Freshwater	Alligator
6	5.0	700	20	Saltwater	Crocodile
7	4.1	550	25	Freshwater	Alligator
8	3.3	420	18	Saltwater	Crocodile
9	4.3	530	24	Freshwater	Alligator

```
# Encode the categorical 'Habitat' and 'Species' columns
label_encoder = LabelEncoder()
data['Habitat'] = label_encoder.fit_transform(data['Habitat'])  #
Convert habitat to numeric
data['Species'] = label_encoder.fit_transform(data['Species'])  #
Convert species to numeric

data
```

	Length	Weight	Snout Width	Habitat	Species
0	3.4	400	20	1	1
1	4.2	500	22	0	0
2	3.9	450	19	1	1
3	4.5	600	24	0	0
4	2.8	300	21	1	1
5	3.6	480	23	0	0
6	5.0	700	20	1	1
7	4.1	550	25	0	0
8	3.3	420	18	1	1
9	4.3	530	24	0	0

```python
# Separate features and target
X = data[['Length', 'Weight', 'Snout Width', 'Habitat']]
y = data['Species']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Standardize the feature values
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize and train the SVM model
svm_model = SVC(kernel='linear', C=1.0)  # Linear kernel SVM for
binary classification
svm_model.fit(X_train, y_train)
```

```
SVC(kernel='linear')
```

```python
# Make predictions on the test set
y_pred = svm_model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred,
target_names=['Alligator', 'Crocodile'])

# Output the results
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

```
Accuracy: 1.0
Classification Report:
               precision    recall  f1-score   support

   Alligator       1.00      1.00      1.00         2
   Crocodile       1.00      1.00      1.00         1

    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```

```python
# Define the habitat mapping outside the function
habitat_mapping = {
    'Freshwater': 0,
    'Saltwater': 1
}

# Define species mapping as well, for completeness
species_mapping = {0: 'Alligator', 1: 'Crocodile'}
```

```python
# Function to predict whether a new input is a Crocodile or Alligator
def predict_species(length, weight, snout_width, habitat):
    # Convert habitat to numeric using the previously saved mapping
    if habitat in habitat_mapping:
        habitat_num = habitat_mapping[habitat]
    else:
        print("Invalid habitat. Please use 'Freshwater' or 
'Saltwater'.")
        return

    # Prepare the feature vector and scale it
    new_data = scaler.transform([[length, weight, snout_width, 
habitat_num]])

    # Predict using the SVM model
    prediction = svm_model.predict(new_data)

    # Map the numeric prediction back to species name
    species_name = species_mapping[prediction[0]]
    print(f"The predicted species is: {species_name}")

# Example input to check the species
# Example: Length=4.0 meters, Weight=500 kg, Snout Width=22 cm, 
Habitat='Freshwater'
predict_species(4.0, 500, 22, 'Freshwater')
```

```
The predicted species is: Alligator

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:464: 
UserWarning: X does not have valid feature names, but StandardScaler 
was fitted with feature names
  warnings.warn(
```

```python
predict_species(3.4, 400, 20, 'Saltwater')
```

```
The predicted species is: Crocodile

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:464: 
UserWarning: X does not have valid feature names, but StandardScaler 
was fitted with feature names
  warnings.warn(
```